

# Principles of Programming Languages

## Type Systems

Andrei Arusoaie<sup>1</sup>

<sup>1</sup>Department of Computer Science

November 28, 2022

# Outline

## Motivation

# Outline

Motivation

Typing

# Outline

Motivation

Typing

Typing rules

# Outline

Motivation

Typing

Typing rules

Properties

# Motivation

```
#include <stdio.h>
int main() {
    int x = 0;
    printf("x = %d, ", x);
    if (x)
    {
        printf("took the 'then' branch\n");
    }
    else
    {
        printf("took the 'else' branch\n");
    }
}
```

# Motivation

Q1: What is the output of this C program?

```
#include <stdio.h>
int main() {
    int x = 0;
    printf("x = %d, ", x);
    if (x)
    {
        printf("took the 'then' branch\n");
    }
    else
    {
        printf("took the 'else' branch\n");
    }
}
```

# Motivation

Q1: What is the output of this C program?

./a.out

x = 0, took the 'else' branch

```
#include <stdio.h>
int main() {
    int x = 0;
    printf("x = %d, ", x);
    if (x)
    {
        printf("took the 'then' branch\n");
    }
    else
    {
        printf("took the 'else' branch\n");
    }
}
```



# Motivation

Q1: What is the output of this C program?

./a.out

x = 0, took the 'else' branch

Q2: What if x is set to 10 instead of 0?

```
#include <stdio.h>
int main() {
    int x = 0;
    printf("x = %d, ", x);
    if (x)
    {
        printf("took the 'then' branch\n");
    }
    else
    {
        printf("took the 'else' branch\n");
    }
}
```

# Motivation

Q1: What is the output of this C program?

```
./a.out
```

```
x = 0, took the 'else' branch
```

Q2: What if x is set to 10 instead of 0?

```
./a.out
```

```
x = 10, took the 'then' branch
```

```
#include <stdio.h>
int main() {
    int x = 0;
    printf("x = %d, ", x);
    if (x)
    {
        printf("took the 'then' branch\n");
    }
    else
    {
        printf("took the 'else' branch\n");
    }
}
```

# Motivation

```
#include <stdio.h>
int main() {
    int x = 0;
    printf("x = %d, ", x);
    if (x)
    {
        printf("took the 'then' branch\n");
    }
    else
    {
        printf("took the 'else' branch\n");
    }
}
```

Boolean and arithmetic expressions are mixed!

# Motivation

Q: What is the type of `x`? What is the type of the first argument of the conditional statement `if` in general?

```
#include <stdio.h>
int main() {
    int x = 0;
    printf("x = %d, ", x);
    if (x)
    {
        printf("took the 'then' branch\n");
    }
    else
    {
        printf("took the 'else' branch\n");
    }
}
```

Boolean and arithmetic expressions are mixed!

# Motivation

- ▶ Mixing Boolean and arithmetic expressions may increase the expressiveness of the language

# Motivation

- ▶ Mixing Boolean and arithmetic expressions may increase the expressiveness of the language

```
if(verbose)  
    printf("Starting first pass\n");
```

# Motivation

- ▶ Mixing Boolean and arithmetic expressions may increase the expressiveness of the language

```
if(verbose)
    printf("Starting first pass\n");
```

- ▶ The code above is still readable if the integer variable `verbose` is conceptually a boolean that holds 0 (false) when the verbose mode is disabled, and a nonzero value (true) when the verbose mode is enabled

# Problem

- ▶ Mixing Boolean and arithmetic expressions may lead to strange expressions:



# Problem

- ▶ Mixing Boolean and arithmetic expressions may lead to strange expressions:

```
#include <stdio.h>
int main() {
    printf("%d\n", 0 < 1 && 3 < 0);
    printf("%d\n", 0 < 1 || 3 < 0);
    printf("%d\n", 0 && 1);
}
```

# Problem

- ▶ Mixing Boolean and arithmetic expressions may lead to strange expressions:

```
#include <stdio.h>
int main() {
    printf("%d\n", 0 < 1 && 3 < 0);
    printf("%d\n", 0 < 1 || 3 < 0);
    printf("%d\n", 0 && 1);
}
```

- ▶ The last expression applies the boolean `and` operator to numbers, not to Boolean expressions
- ▶ Conceptually, the last expression is *ill-formed*

# Let us mix our expressions in Coq

```
Inductive Exp :=  
| anum : nat -> Exp  
| avar : Var -> Exp  
| aplus : Exp -> Exp -> Exp  
| amul : Exp -> Exp -> Exp  
| btrue : Exp  
| bfalse : Exp  
| blessthan : Exp -> Exp -> Exp  
| bnot : Exp -> Exp  
| band : Exp -> Exp -> Exp.
```

```
Coercion anum : nat >-> Exp.
```

```
Coercion avar : Var >-> Exp.
```

```
Notation "A + ' B" := (aplus A B) (at level 50).
```

```
Notation "A * ' B" := (amul A B) (at level 46).
```

```
Notation "A <= ' B" := (blessthan A B) (at level 53).
```

# Let us mix our expressions in Coq

```
Eval compute in (2 +' 2) .  
  = 2 +' 2  
   : Exp
```

```
Eval compute in (2 +' btrue) .  
  = 2 +' btrue  
   : Exp
```

```
Eval compute in (band 2 2) .  
  = band 2 2  
   : Exp
```

# Semantics

```
Inductive exp_eval_small_step :  
  Exp -> State -> Exp -> Prop :=  
| econst : forall n st, anum n =[ st ]=> n  
| elookup : forall v st, avar v =[ st ]=> (st v)  
| eadd_1 : forall a1 a2 a1' st,  
  a1 =[ st ]=> a1' ->  
  a1 +' a2 =[ st ]=> a1' +' a2  
| eadd_2 : forall a1 a2 a2' st,  
  a2 =[ st ]=> a2' ->  
  a1 +' a2 =[ st ]=> a1 +' a2'  
| eadd : forall i1 i2 st n,  
  n = anum (i1 + i2) ->  
  anum i1 +' anum i2 =[ st ]=> n  
...  
| enot : forall b b' state,  
  b =[ state ]=> b' ->  
  (bnot b) =[ state ]=> (bnot b')  
| enottrue : forall state,  
  (bnot btrue) =[ state ]=> bfalse  
| enotfalse : forall state,  
  (bnot bfalse) =[ state ]=> btrue  
...
```

# Well-typed expressions

```
Reserved Notation "A =[ S ]>* A'" (at level 60).  
Inductive aeval_clos : Exp -> State -> Exp -> Prop :=  
| e_refl : forall a st, a =[ st ]>* a  
| e_trans : forall a1 a2 a3 st,  
            (a1 =[st]=> a2) ->  
            a2 =[ st ]>* a3 ->  
            a1 =[ st ]>* a3  
where "A =[ S ]>* A'" := (aeval_clos A S A').
```

```
Example e2 :  
  2 +' x =[ signal ]>* anum 12.
```

**Proof.**

```
apply e_trans with (a2 := (2 +' 10)).  
- apply eadd_2.  
  apply elookup.  
- eapply e_trans.  
  + eapply eadd. eauto.  
  + simpl. eapply e_refl.
```

**Qed.**

- Note that we applied steps until we obtained a **value**

# Ill-typed expressions

**Example** `ill_typed :`

```
2 +' btrue =[ signal ]>* ?.
```

- ▶ It is impossible to obtain a **value** using `exp_eval_small_step`;
- ▶ In such cases we say that `2 +' btrue` is **stuck**:  
`2 +' btrue` is not a value and no transitions are available from it

► Types: Bool and Nat

► Coq:

```
Inductive Typ : Type :=  
  | Bool : Typ  
  | Nat : Typ.
```

► We define a relation  $:\subseteq \text{Exp} \times \text{Typ}$  using inference rules for typing

► When  $e : \text{Nat}$  we say that the type of  $e$  is Nat



# Typing rules - I

$$\frac{n \in \mathbb{N}}{n : \text{Nat}} \text{ TNUM}$$

$$\frac{x \in \text{Var}}{x : \text{Nat}} \text{ TVAR}$$

$$\frac{a_1 : \text{Nat} \quad a_2 : \text{Nat}}{a_1 +' a_2 : \text{Nat}} \text{ TPLUS}$$

$$\frac{a_1 : \text{Nat} \quad a_2 : \text{Nat}}{a_1 *' a_2 : \text{Nat}} \text{ TMUL}$$

## Typing rules - example

$$\frac{\frac{2 \in \mathbb{N}}{2 : \text{Nat}} \text{ TNUM} \quad \frac{x \in \text{Var}}{x : \text{Nat}} \text{ TVAR}}{2 +' x : \text{Nat}} \text{ TPLUS}$$

## Typing rules - II

$$\frac{\cdot}{btrue : Bool} \text{ TTRUE}$$

$$\frac{\cdot}{bfalse : Bool} \text{ TFALSE}$$

$$\frac{a_1 : Nat \quad a_2 : Nat}{a_1 \leq' a_2 : Bool} \text{ TLEQ}$$

$$\frac{b : bool}{bnot \ b : Bool} \text{ TNOT}$$

$$\frac{b_1 : Bool \quad b_2 : Bool}{band \ b_1 \ b_2 : Bool} \text{ TAND}$$

# Typing rules - example

$$\frac{\frac{2 \in \mathbb{N}}{2 : \text{Nat}} \text{ TNUM} \quad \frac{x \in \text{Var}}{x : \text{Nat}} \text{ TVAR}}{2 \leq' x : \text{Bool}} \text{ TLEQ}$$

# Typing rules - I - in Coq

```
Inductive type_of : Exp -> Typ -> Prop :=  
| typ_num : forall n, type_of (anum n) Nat  
| typ_var : forall x, type_of (avar x) Nat  
| typ_plus : forall a1 a2,  
               (type_of a1 Nat) ->  
               (type_of a2 Nat) ->  
               (type_of (a1 +' a2) Nat)  
| typ_mul : forall a1 a2,  
               (type_of a1 Nat) ->  
               (type_of a2 Nat) ->  
               (type_of (a1 *' a2) Nat)  
  
...
```

## Typing rules - II - in Coq

```
Inductive type_of : Exp -> Typ -> Prop :=  
  ...  
| typ_true : type_of btrue Bool  
| typ_false : type_of bfalse Bool  
| typ_lessthan : forall a1 a2,  
    (type_of a1 Nat) ->  
    (type_of a2 Nat) ->  
    (type_of (a1 <= ' a2) Bool)  
| typ_not : forall b,  
    (type_of b Bool) ->  
    (type_of (bnot b) Bool)  
| typ_and : forall b1 b2,  
    (type_of b1 Bool) ->  
    (type_of b2 Bool) ->  
    (type_of (band b1 b2) Bool) .
```

# Values

```
Inductive nat_value : Exp -> Prop :=  
| nat_val : forall n, nat_value (anum n).
```

```
Inductive b_value : Exp -> Prop :=  
| b_true : b_value btrue  
| b_false : b_value bfalse.
```

```
Definition value (e : Exp) :=  
  nat_value e \/ b_value e.
```

# Canonical forms

```
Lemma bool_canonical :  
  forall e, type_of e Bool ->  
    value e ->  
    b_value e.
```

```
Lemma nat_canonical :  
  forall e, type_of e Nat ->  
    value e ->  
    nat_value e.
```



# Progress

**Theorem** progress :  
 forall t T state,  
 type\_of t T ->  
 value t \ / **exists** t', t =[ state ]=> t'.

**Proof.**

*(\* By induction on type\_of \*)*

**Admitted.**

# Type preservation

**Theorem** preservation :  
  **forall** t T t' state,  
    type\_of t T ->  
    t =[ state ]=> t' ->  
    type\_of t' T.

**Proof.**

*(\* By induction on type\_of \*)*

**Admitted.**

# Type soundness

**Corollary** soundness :  
  **forall** t t' state T,  
    type\_of t T ->  
    t =[ state ]>\* t' ->  
    value t' \ / **exists** t'', t' =[ state ]=> t''.

**Proof.**

*(\* By induction on the steps relation \*)*

**Admitted.**

# Resources

1. Software Foundations - Volume 2, Benjamin C. Pierce, Arthur Azevedo de Amorim, Chris Casinghino, Marco Gaboardi, Michael Greenberg, Cătălin Hrițcu, Vilhelm Sjöberg, Andrew Tolmach, Brent Yorgey

Section “*Type Systems*”:

[https://softwarefoundations.cis.upenn.edu/  
current/plf-current/Types.html](https://softwarefoundations.cis.upenn.edu/current/plf-current/Types.html)